

EduScore MLOps

Technical Documentation

End-to-End DataOps & MLOps Pipeline

Project: EduScore — Student Performance Predictor

Version: 0.1.0

Stack: Python 3.12, DuckDB, dbt, dlt, Dagster,
MLflow, FastAPI, Docker

Dataset: Kaggle — Student Performance Dataset

Date: July 11, 2026

Contents

1	Project Overview	3
1.1	Introduction	3
1.2	Problem Statement	3
1.3	Technology Stack	4
2	System Architecture	5
2.1	High-Level Architecture	5
2.2	Repository Structure	6
3	DataOps Pipeline	7
3.1	Overview	7
3.2	Data Versioning (DVC)	7
3.3	Ingestion with dlt	8
3.4	Transformations with dbt	8
4	Data Contract	9
4.1	Canonical Schema	9
4.2	Source Aliases	9
4.3	Grade Derivation	10
4.4	Quality Checks	10
5	MLOps Pipeline	11
5.1	Dagster Asset Graph	11
5.2	Dataset Fingerprinting and Conditional Training	12
6	Machine Learning Model	13
6.1	Model Architecture	13
6.2	Evaluation Metrics	13
6.3	Quality Gate	14
6.4	MLflow Experiment Tracking	14

7	API Serving	16
7.1	FastAPI Application	16
7.2	Endpoints	16
7.3	Example Request	16
7.4	Request Lifecycle	17
8	Monitoring	18
8.1	Prediction Logging	18
8.2	Drift Detection	18
8.3	Recommended Production Signals	18
9	Infrastructure and CI/CD	19
9.1	Docker	19
9.2	GitHub Actions CI	19
9.3	Environment Variables	20
10	Running the Project	21
10.1	Prerequisites	21
10.2	Setup	21
10.3	Full Local Pipeline	21
10.4	Dagster	22
10.5	Pre-Push Checks	22
11	Troubleshooting	23
12	Course Requirements Mapping	24
A	Git Commit Policy	25

Chapter 1

Project Overview

1.1 Introduction

EduScore MLOps is an end-to-end Machine Learning Operations (MLOps) and Data Operations (DataOps) project that predicts a student’s final academic score from three behavioral signals. The project demonstrates the full production lifecycle of a machine learning system: data ingestion, warehousing, transformation, validation, model training, experiment tracking, API serving, monitoring, orchestration, containerization, and continuous integration.

The raw dataset is the Student Performance Dataset from Kaggle.¹ It is not committed to Git but tracked by DVC and stored in Google Drive.

1.2 Problem Statement

The model accepts three input features:

- `study_hours` — weekly self-study hours (0–40)
- `attendance_percentage` — attendance rate (0–100%)
- `class_participation` — participation score (0–10)

It produces two outputs:

- `total_score` — predicted numeric score in $[0, 100]$
- `grade` — derived letter grade (A/B/C/D/F)

¹<https://www.kaggle.com/datasets/nabeelqureshitiii/student-performance-dataset>

1.3 Technology Stack

Table 1.1 summarizes the tools used across every layer of the project.

Table 1.1: Technology stack

Layer	Tool	Role
Data versioning	DVC + Google Drive	Raw CSV versioning
Ingestion	dlt	CSV → DuckDB
Warehouse	DuckDB	Local analytical DB
Transformation	dbt	SQL models and tests
Orchestration	Dagster	Asset graph and schedule
ML training	scikit-learn	RandomForestRegressor
Experiment track.	MLflow	Params, metrics, registry
API serving	FastAPI + Uvicorn	REST prediction endpoint
Monitoring	Custom JSONL logger	Prediction drift logging
Containerization	Docker + Compose	Reproducible deployment
CI	GitHub Actions	Lint, test, Docker build

Chapter 2

System Architecture

2.1 High-Level Architecture

Figure 2.1 shows the end-to-end data and model flow. Dagster orchestrates the ingestion, validation, and training stages. The fingerprint decision determines whether training runs or is skipped.

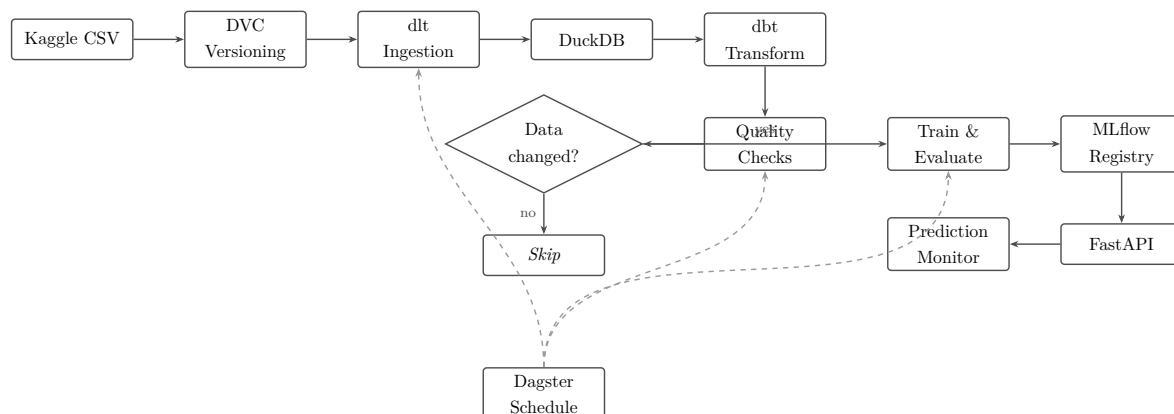


Figure 2.1: High-level system architecture

2.2 Repository Structure

Table 2.1: Repository layout

Path	Purpose
api/	FastAPI application
data/raw/	DVC-tracked raw CSV pointer
dbt/student_score/	dbt project (models, tests, profiles)
dlt_pipeline/	dlt ingestion script
docs/	Project documentation
monitoring/	Prediction log schema
pipelines/	Dagster asset graph and schedule
scripts/	Windows helpers
src/student_score_mlops/	Config, data, training, prediction, monitoring
tests/	pytest suite
.github/workflows/	GitHub Actions CI

Chapter 3

DataOps Pipeline

3.1 Overview

The DataOps layer transforms the raw CSV through five sequential stages before the data reaches the ML layer. Figure 3.1 illustrates this flow.

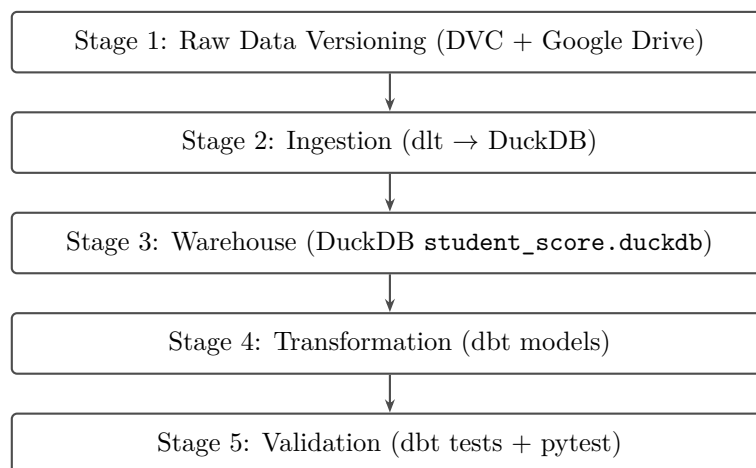


Figure 3.1: DataOps pipeline stages

3.2 Data Versioning (DVC)

The raw CSV is never committed to Git. DVC tracks its hash via the pointer file `data/raw/student_performance.csv.dvc`. Restoring the data:

```
1 dvc pull                                # macOS/Linux
2 scripts\dvc.cmd pull                    # Windows
```

Listing 3.1: Restore raw data

3.3 Ingestion with dlt

The script `dlt_pipeline/load_student_data.py` reads the CSV and loads it into DuckDB, normalising column names to the canonical schema (Chapter 4).

3.4 Transformations with dbt

The dbt project at `dbt/student_score/` builds the view `ml_student_training_set`. It applies column aliasing, type casting, and row-level filtering.

```
1 dbt run --project-dir dbt/student_score --profiles-dir dbt/  
  student_score  
2 dbt test --project-dir dbt/student_score --profiles-dir dbt/  
  student_score
```

Listing 3.2: Run dbt

Chapter 4

Data Contract

4.1 Canonical Schema

Table 4.1: Data contract schema

Column	Type	Required	Constraint
study_hours	float	Yes	$0 \leq x \leq 40$
attendance_percentage	float	Yes	$0 \leq x \leq 100$
class_participation	float	Yes	$0 \leq x \leq 10$
total_score	float	Training only	$0 \leq x \leq 100$
grade	string	Optional	$\in \{A, B, C, D, F\}$

4.2 Source Aliases

Table 4.2: Accepted column name aliases

Canonical	Accepted Aliases
study_hours	average_weekly_self_study_hours, weekly_self_study_hours, hours_studied
attendance_percentage	attendance, attendance_rate
class_participation	participation, participation_score
total_score	final_score, score, performance_score
grade	final_grade

4.3 Grade Derivation

Table 4.3: Grade assignment rule

Score Range	Grade
$\text{score} \geq 85$	A
$70 \leq \text{score} < 85$	B
$55 \leq \text{score} < 70$	C
$40 \leq \text{score} < 55$	D
$\text{score} < 40$	F

4.4 Quality Checks

Data quality is enforced at three levels before training is allowed to proceed (Figure 4.1).

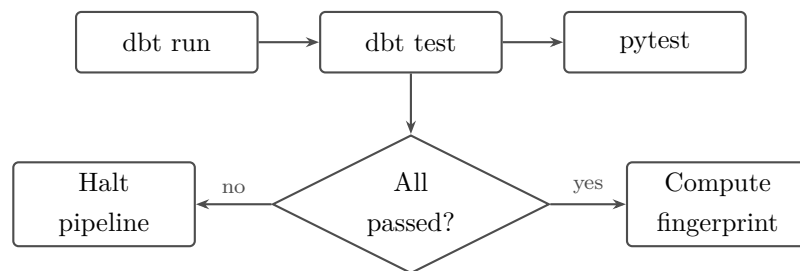


Figure 4.1: Data quality gate

Chapter 5

MLOps Pipeline

5.1 Dagster Asset Graph

All pipeline assets are defined in `pipelines/dagster_pipeline.py`. Figure 5.1 shows the dependency chain. A daily schedule (`daily_student_score_pipeline`) runs at 02:00 Africa/Casablanca time.

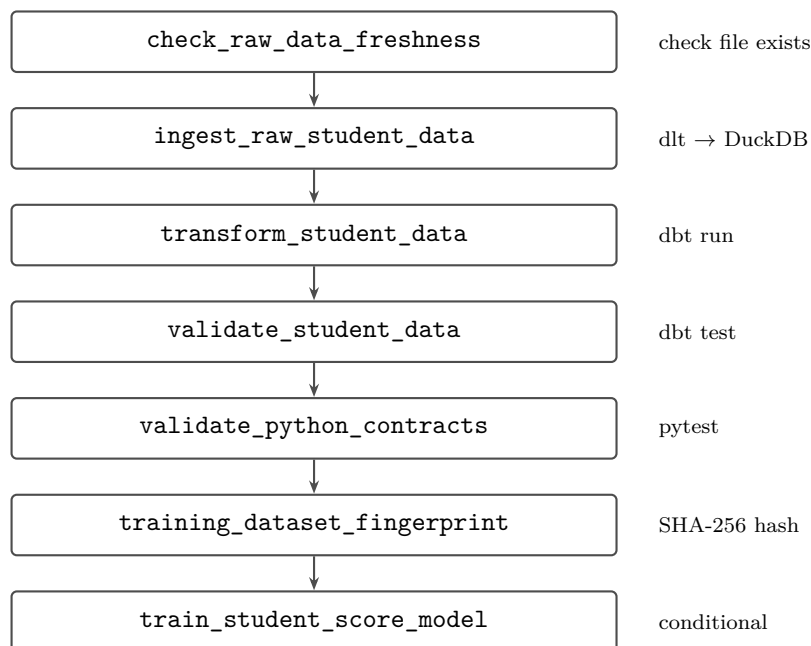


Figure 5.1: Dagster asset dependency graph

5.2 Dataset Fingerprinting and Conditional Training

After validation, Dagster computes a deterministic SHA-256 fingerprint of the training dataset. The fingerprint is compared with the last recorded value in `metadata/data_fingerprint.json`. Training runs only when the fingerprint has changed or `force_train` is set. Figure 5.2 shows this logic.

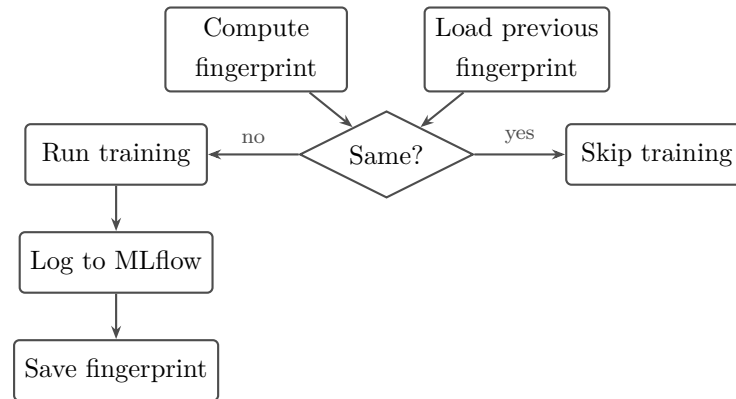


Figure 5.2: Fingerprint-based conditional retraining

To force retraining via Dagster run config:

```
1 ops:
2   train_student_score_model:
3     config:
4       force_train: true
```

Listing 5.1: Force-train config

Chapter 6

Machine Learning Model

6.1 Model Architecture

The training script (`src/student_score_mlops/train.py`) builds a scikit-learn `Pipeline` with two stages:

1. **StandardScaler** — zero-mean, unit-variance normalisation.
2. **RandomForestRegressor** — ensemble of 200 decision trees.

Table 6.1: Model hyperparameters

Parameter	Value
<code>n_estimators</code>	200
<code>max_depth</code>	8
<code>random_state</code>	42
<code>n_jobs</code>	−1 (all cores)
Test split	20%

6.2 Evaluation Metrics

Three metrics are computed on the held-out test set:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (6.1)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (6.2)$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (6.3)$$

6.3 Quality Gate

Training passes only when both thresholds in Table 6.2 are met. Otherwise a `RuntimeError` is raised and the model is not registered. Figure 6.1 shows the decision logic.

Table 6.2: Quality thresholds

Metric	Threshold	Env Variable
R^2	≥ 0.70	MIN_R2
RMSE	≤ 15.0	MAX_RMSE

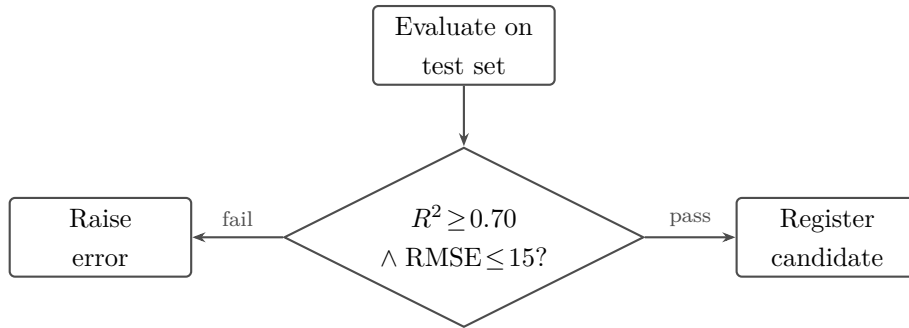


Figure 6.1: Model quality gate

6.4 MLflow Experiment Tracking

Each training run logs to the local `mlruns/` directory:

- **Parameters:** model type, hyperparameters, dataset fingerprint, row count, thresholds, training timestamp.
- **Metrics:** MAE, RMSE, R^2 , `passed_thresholds`.
- **Tags:** dataset fingerprint, training timestamp, quality gate result.

- **Artifacts:** `model.joblib`, `metrics.json`, `training_reference.json`.

If metrics pass, the model is registered as `EduScoreStudentPerformanceModel`. Registration is not deployment; promotion to production requires a separate approval step.

Chapter 7

API Serving

7.1 FastAPI Application

The REST API (`api/main.py`) loads the trained model on startup.

```
1 uvicorn api.main:app --host 0.0.0.0 --port 8000
```

Listing 7.1: Start the API

7.2 Endpoints

Table 7.1: API endpoints

Method	Path	Description
GET	/health	Liveness check
GET	/model-info	Model path, metrics, fingerprint, timestamp
POST	/predict	Predict score and grade

7.3 Example Request

```
1 curl -X POST http://localhost:8000/predict \  
2   -H "Content-Type: application/json" \  
3   -d '{"study_hours": 12,  
4       "attendance_percentage": 88,  
5       "class_participation": 7}'
```

Listing 7.2: Prediction request

```
1 {"predicted_score": 78.42, "predicted_grade": "B"}
```

Listing 7.3: Example response

7.4 Request Lifecycle

Figure 7.1 shows the flow of a single prediction request.

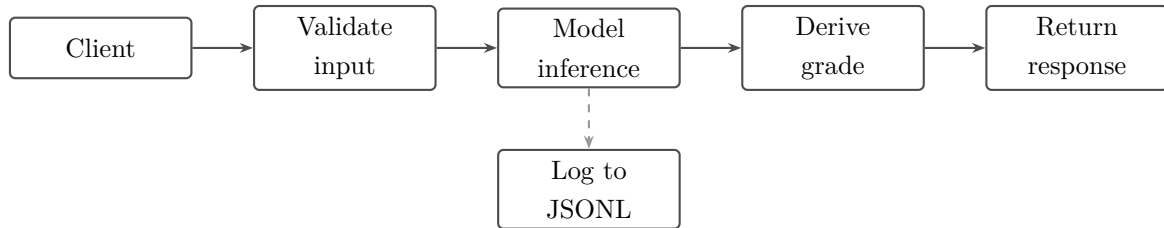


Figure 7.1: Prediction request lifecycle

Chapter 8

Monitoring

8.1 Prediction Logging

Every prediction is appended as a JSON line to `monitoring/predictions.jsonl`. Each record contains the input features, predicted score and grade, latency in milliseconds, and an ISO-8601 timestamp.

8.2 Drift Detection

The monitoring module (`src/student_score_mlops/monitoring.py`) compares recent prediction feature means against the training reference statistics in `models/training_reference.json` enabling lightweight input drift detection.

8.3 Recommended Production Signals

Table 8.1: Production monitoring signals

Signal	Description
Request latency	P50/P95/P99 response times
Error rates	4xx/5xx HTTP error percentages
Input drift	Feature distribution shift vs. training reference
Prediction drift	Output score distribution shift
Label feedback	Actual scores vs. predictions when available
Data freshness	Time since last successful dbt run

Chapter 9

Infrastructure and CI/CD

9.1 Docker

```
1 docker compose up --build api          # API only
2 docker compose up --build             # API + MLflow
```

Listing 9.1: Docker Compose

The API container mounts local `models/` and `monitoring/`. Services are exposed at `http://localhost:8000` (API) and `http://localhost:5000` (MLflow UI).

9.2 GitHub Actions CI

Figure 9.1 shows the CI pipeline. GitHub Actions handles code quality; Dagster handles retraining.

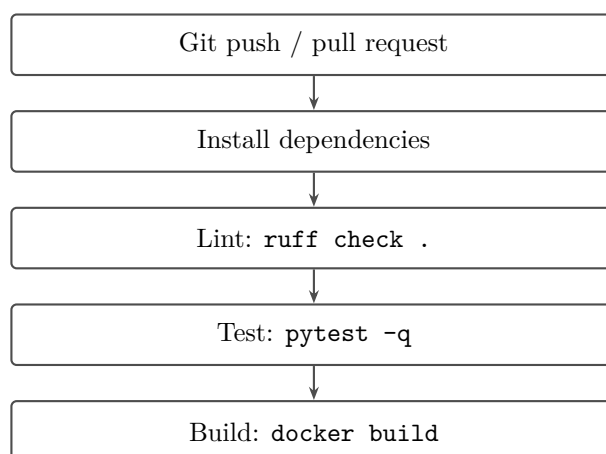


Figure 9.1: GitHub Actions CI pipeline

9.3 Environment Variables

Table 9.1: Key environment variables

Variable	Default	Purpose
RAW_DATA_PATH	data/raw/student_performance.csv	Input CSV
DUCKDB_PATH	data/student_score.duckdb	Warehouse
MODEL_PATH	models/...model.joblib	Model artifact
MLFLOW_TRACKING_URI	mlruns	Tracking dir
MIN_R2	0.70	Quality gate
MAX_RMSE	15.0	Quality gate

Chapter 10

Running the Project

10.1 Prerequisites

Python 3.12, Git, and optionally Docker Desktop. A Google account with access to the DVC Google Drive folder is needed to restore the raw data.

10.2 Setup

```
1 git clone <repo-url> && cd eduscore-mlops
2 python -m venv .venv && source .venv/bin/activate
3 pip install --upgrade pip && pip install -r requirements.txt
4 dvc pull
```

Listing 10.1: Setup (macOS/Linux)

```
1 git clone <repo-url>; cd eduscore-mlops
2 python -m venv .venv; .\.venv\Scripts\Activate.ps1
3 pip install --upgrade pip; pip install -r requirements.txt
4 scripts\dvc.cmd pull
```

Listing 10.2: Setup (Windows PowerShell)

10.3 Full Local Pipeline

```
1 python dlt_pipeline/load_student_data.py
2 dbt run --project-dir dbt/student_score --profiles-dir dbt/
  student_score
```

```
3 dbt test --project-dir dbt/student_score --profiles-dir dbt/  
  student_score  
4 python -m src.student_score_mlops.train
```

Listing 10.3: Manual pipeline run

10.4 Dagster

```
1 scripts\dagster_dev.cmd
```

Listing 10.4: Start Dagster

Open the Dagster UI and materialize the EduScore assets.

10.5 Pre-Push Checks

```
1 ruff check .  
2 pytest -q  
3 dbt test --project-dir dbt/student_score --profiles-dir dbt/  
  student_score  
4 scripts\dvc.cmd status
```

Listing 10.5: Quality checks

Chapter 11

Troubleshooting

Table 11.1: Common issues

Error	Resolution
<code>ModuleNotFoundError: pkg_resources</code>	<code>pip install -r requirements.txt</code> (pins <code>setuptools<81</code>).
Google: “This app is blocked”	Create a custom OAuth Desktop client; configure with <code>scripts\setup_gdrive_oauth.cmd</code> .
Error 403: <code>access_denied</code>	Add your account as a Test User in Google Cloud Console.
DVC writes outside workspace	Use <code>scripts\dvc.cmd</code> wrapper.
API: model file missing	Run <code>python -m src.student_score_mlops.train</code> .

Chapter 12

Course Requirements Mapping

Table 12.1: Requirement to implementation mapping

Requirement	Implementation
Data Strategy	docs/vision.md, DVC-tracked dataset
Agile	docs/agile.md
dlt	dlt_pipeline/load_student_data.py
DuckDB	data/student_score.duckdb
dbt	dbt/student_score/
Dagster	pipelines/dagster_pipeline.py
Data Quality	docs/data_contract.md, pytest, dbt tests
ML	src/student_score_mlops/train.py
MLflow	Experiment tracking in training script
FastAPI	api/main.py
Docker	Dockerfile, docker-compose.yml
CI/CD	.github/workflows/ci.yml
Monitoring	monitoring/, monitoring.py

Appendix A

Git Commit Policy

Commit: source code, tests, docs, Docker files, dbt files, Dagster files, `.dvc/config`, `.dvcignore`, `data/raw/student_performance.csv.dvc`, and helper scripts.

Do not commit: `.dvc/config.local`, `.venv/`, `data/raw/*.csv`, `data/*.duckdb`, `models/*.joblib`, `models/*.json`, `metadata/*.json`, `mlruns/`, `monitoring/*.jsonl`, and Dagster/dlt cache folders.